

Estrutura de Dados

Pilhas

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Uma Única Porta

As listas encadeadas das Aulas 04 e 05 permitiam inserir e remover em qualquer posição. A pilha é a mesma ideia de nós ligados por ponteiros, sob uma restrição deliberada: existe uma única porta de entrada e saída, o topo. Nenhum nó do meio ou do fim é jamais tocado diretamente — só o que está no topo.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

LIFO: o Último a Entrar É o Primeiro a Sair

Empilhar é inserir acima do topo atual. Desempilhar é remover o topo e revelar o nó que estava logo abaixo dele. Essa ordem — o último elemento inserido é sempre o primeiro a ser removido — chama-se LIFO, e é o comportamento inteiro da pilha: não existe outra forma de entrar ou sair.



Prof. Everaldo Graciliano Souza Ribeiro

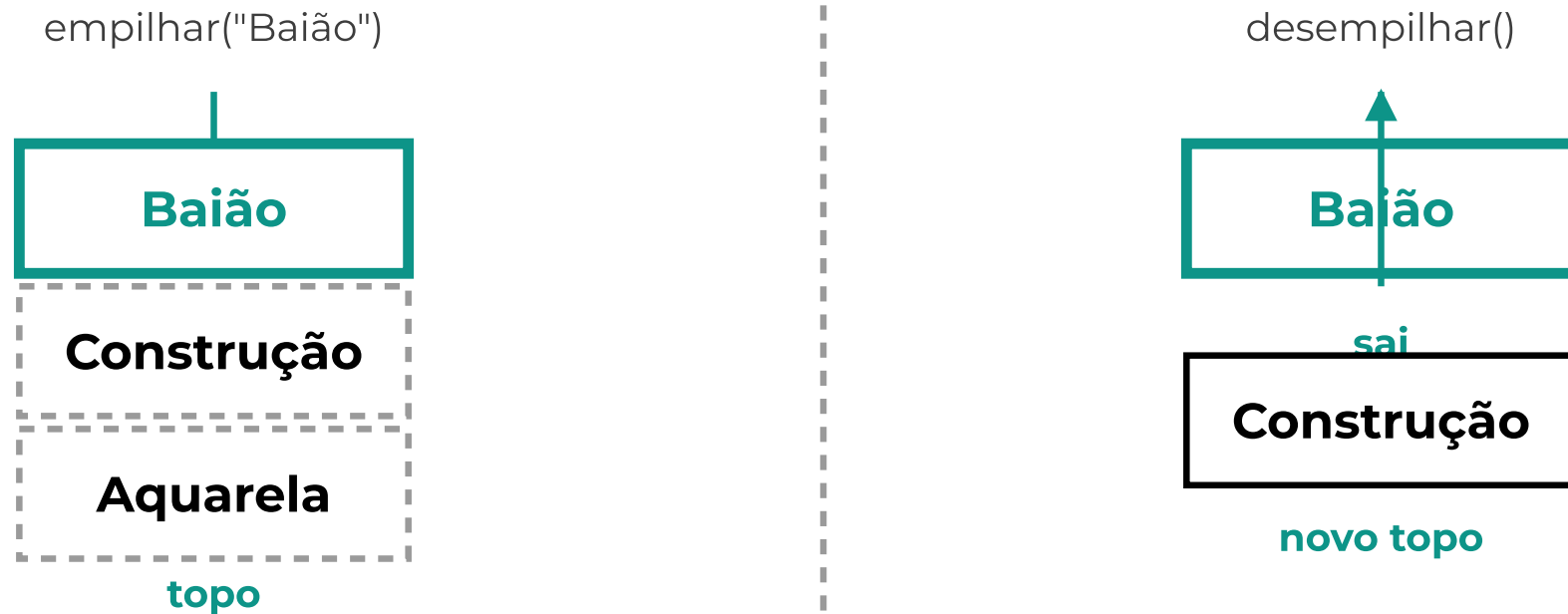
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Empilhar e Desempilhar



Os dois lados nunca são tocados. **Só o topo muda** — por isso o custo é sempre constante.



Custo Constante, dos Dois Lados

Empilhar cria um nó e o liga ao antigo topo — nenhum outro nó da pilha é percorrido. Desempilhar troca o topo pelo nó de baixo, que já estava ligado a ele. As duas operações custam o mesmo, independentemente de a pilha ter três elementos ou três mil, porque nenhuma delas precisa caminhar pela estrutura.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Pilha Não É Fila

A pilha resolve um problema estruturalmente oposto ao da fila que aparecerá na Aula 07. A fila serve quando a ordem de saída deve reproduzir a ordem de chegada. A pilha serve quando o mais recente é o que importa primeiro — desfazer uma ação, voltar uma tela, visitar o que acabou de acontecer. Escolher entre as duas é consequência da regra do problema, não de preferência.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Histórico de Reprodução no SoundLab

O SoundLab ganha uma pilha para representar o histórico de reprodução: cada faixa tocada diretamente do acervo é empilhada, e o topo é sempre a faixa que está tocando agora. Desfazer a reprodução desempilha e revela o que tocava antes dela — o botão "voltar" de qualquer tocador de música.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Histórico Não É o Cursor da Playlist

A Aula 05 já deu à Playlist um jeito de voltar uma faixa: o cursor anda pelo ponteiro anterior, sempre dentro da ordem da playlist. O histórico é outra estrutura, para outro problema: ele guarda a ordem real em que as faixas foram tocadas, mesmo quando uma faixa é tocada direto do acervo, fora de qualquer posição da playlist — algo que o cursor nunca poderia reconstruir sozinho.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Nó da Pilha

Cada elo aponta so para o no de baixo. Nao existe ponteiro
para cima: a pilha nunca precisa subir, so descer a partir do topo.

```
class _No:
    def __init__(self, valor, abaixo=None):
        self.valor = valor
        self.abaixo = abaixo
```

```
class Pilha:
    def __init__(self):
        self._topo = None
        self._tamanho = 0
```



Empilhar e Desempilhar

```
def empilhar(self, valor):  
    # Custo constante: nenhum outro no da pilha e visitado.  
    self._topo = _No(valor, abaixo=self._topo)  
    self._tamanho += 1  
  
def desempilhar(self):  
    if self.vazia():  
        raise IndexError("pilha vazia")  
    no = self._topo  
    self._topo = no.abaixo      # o no de baixo vira o novo topo  
    self._tamanho -= 1  
    return no.valor
```



Tocar Agora: Empilhar uma Reprodução

```
def tocar_agora(acervo, historico):  
    # Toca direto do acervo, fora da ordem da playlist.  
    faixa = acervo.buscar(input("  titulo: ").strip())  
    if faixa is None:  
        print("  nao esta no acervo")  
        return  
    historico.empilhar(faixa)  
    print(f"  tocando agora: {faixa}")
```



Desfazer: Desempilhar a Reprodução

```
def desfazer_reproducao(historico):  
    if historico.vazia():  
        print("  nada no historico")  
        return  
    atual = historico.desempilhar()  
    print(f"  saindo de: {atual}")  
    if historico.vazia():  
        print("  historico vazio - nada tocando")  
    else:  
        print(f"  voltando a tocar: {historico.topo()}")
```



Atividades

Exercício 1

Se o histórico tem Aquarela, depois Baião, depois Construção empilhadas nessa ordem, o que aparece ao desempilhar duas vezes?

Exercício 2

Por que _No da pilha só precisa de um ponteiro, enquanto o nó da lista duplamente encadeada da Aula 05 precisa de dois?

Exercício 3

Por que o histórico de reprodução não poderia ser reconstruído usando só o cursor anterior da Playlist?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Criar esvaziar() na Pilha, desempilhando tudo de uma vez.
2. Fazer o menu mostrar quantas faixas existem no histórico antes de cada reprodução.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>